

AI-Augmented SOC Implementation for Enterprise Threat Detection and Response

Course-End Project Writeup

Simplilearn / IIIT-Bangalore

Advanced Executive Program in Cybersecurity

Course-End Project

AkshayKumar Hiremath

Submitted: 12 July 2026

Table of Contents

1. Environment Setup
2. Splunk Enterprise Installation (Ubuntu)
3. Windows Log Forwarding Setup
 - 3.1 Sysmon Installation
 - 3.2 Splunk Universal Forwarder
 - 3.3 Sysmon Ingestion Gap and Fix
4. Splunk Boot Persistence
5. Elastic Stack Installation (Ubuntu)
6. Elastic Agent Deployment (Windows VM)
7. LogAI AI-Driven Log Anomaly Detection
8. Task 2 AI-Driven Detection and Threat Intelligence
 - 8.1 Elastic ML Trial License
 - 8.2 Elastic ML Anomaly Detection Jobs
 - 8.3 Aikido AI-Driven Application Security
 - 8.4 Sophos Intercept X Endpoint Protection with AI
 - 8.5 OpenCTI Threat Intelligence Platform
9. Task 3 Threat Hunting and IOC Pivoting
 - 9.1 Attack Traffic Generation
 - 9.2 Enabling Firewall Visibility
 - 9.3 IOC Hunt Source 1: Windows Firewall Log
 - 9.4 IOC Hunt Source 2: Windows Security Event Log
 - 9.5 Cross-Source Pivot Summary
 - 9.6 Key Observations from the Hunt
10. Task 4 SOAR Automation, MITRE/D3FEND Mapping, and Gap Analysis
 - 10.1 Shuffle Deployment
 - 10.2 Playbook Design - Failed Logon Auto-Response
 - 10.3 Deployment Challenges Documented
 - 10.4 MITRE ATT&CK Coverage Mapping
 - 10.5 D3FEND Countermeasure Mapping
 - 10.6 SOAR Playbook Execution Evidence
 - 10.7 Gap Analysis
 - 10.8 Prioritized Recommendations
 - 10.9 Task 4 Conclusion

Decisions Log

1. Environment Setup

I built the lab on three VMs in VMware Workstation Pro, all on the same host-only/NAT segment so they could reach each other cleanly. I split the roles deliberately: keep the noisy attacker isolated, keep the endpoint clean, and give the SIEM host the memory headroom it needs.

- **Ubuntu Server (CLI-only)** : SIEM stack (Splunk Enterprise + Elastic Stack), later hosting Docker for OpenCTI and Shuffle.
- **Windows 11 Enterprise**: monitored endpoint. Sysmon, Splunk Universal Forwarder, Elastic Agent, and Sophos Intercept X all installed here.
- **Kali Linux**: attacker VM, kept dormant until Task 3.

Splunk and Elasticsearch are both memory-heavy in tandem, so I rebalanced RAM across the VMs before starting Task 1: I dropped Kali from 12 GB to 6 GB (idle until Task 3 anyway) and pushed Ubuntu up to around 16 GB to give the SIEM services enough headroom.

VM inventory:

VM	Role	IP Address
Ubuntu Server	SIEM host (Splunk + Elastic + Docker stacks)	192.168.127.141
Windows 11 Enterprise	Monitored endpoint (Sysmon, UF, Elastic Agent, Sophos)	192.168.127.134
Kali Linux	Attacker VM (Task 3 recon + credential attack)	192.168.127.131

2. Splunk Enterprise Installation (Ubuntu)

I installed Splunk via the official .deb package and ran through the first-launch setup interactively to create the admin credentials.

What went wrong: Splunk kept failing to start. All I got was a deprecation warning ("Running Splunk Enterprise as root is deprecated") before the shell dropped back to the prompt, no splunkd.log ever written, no error, nothing useful.

After ruling out permissions and disk, I traced it to a behaviour change in Splunk 10.4.1: running as root without acknowledging it explicitly aborts the startup silently rather than just warning. I fixed it by passing the flag:

```
sudo /opt/splunk/bin/splunk start --accept-license --run-as-root
```

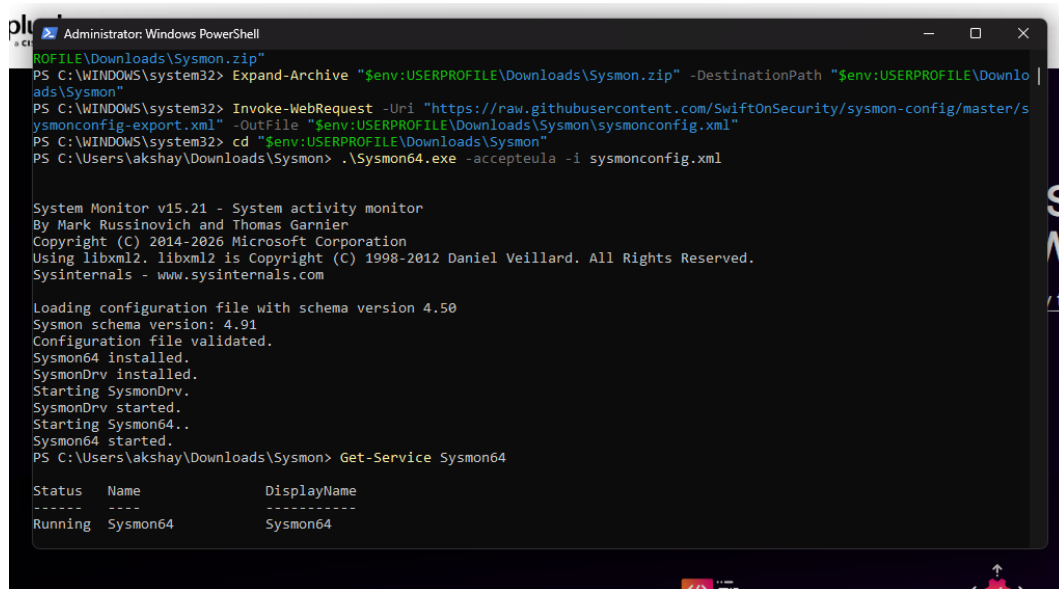
After that Splunk started cleanly and the web interface came up.

Networking note: the Ubuntu VM's NAT config didn't allow my physical host machine to reach the Splunk web UI directly, so I accessed it from the Windows VM's browser instead. That's functionally the same pattern a real SOC analyst would use reach the SIEM from a workstation on the internal network, not from outside.

3. Windows Log Forwarding Setup

3.1 Sysmon Installation

I installed Sysmon on the Windows 11 VM using the SwiftOnSecurity configuration template. That gets me process creation, network connection, and file integrity telemetry beyond what the default Windows event logs give up. Service confirmed running:



```
Administrator: Windows PowerShell
PS C:\Users\akshay\Downloads\Sysmon> Expand-Archive "$env:USERPROFILE\Downloads\Sysmon.zip" -DestinationPath "$env:USERPROFILE\Downloads\Sysmon"
PS C:\WINDOWS\system32> Invoke-WebRequest -Uri "https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.xml" -OutFile "$env:USERPROFILE\Downloads\Sysmon\sysmonconfig.xml"
PS C:\WINDOWS\system32> cd "$env:USERPROFILE\Downloads\Sysmon"
PS C:\Users\akshay\Downloads\Sysmon> .\Sysmon64.exe -accepteula -i sysmonconfig.xml

System Monitor v15.21 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.50
Sysmon schema version: 4.91
Configuration file validated.
Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.
PS C:\Users\akshay\Downloads\Sysmon> Get-Service Sysmon64

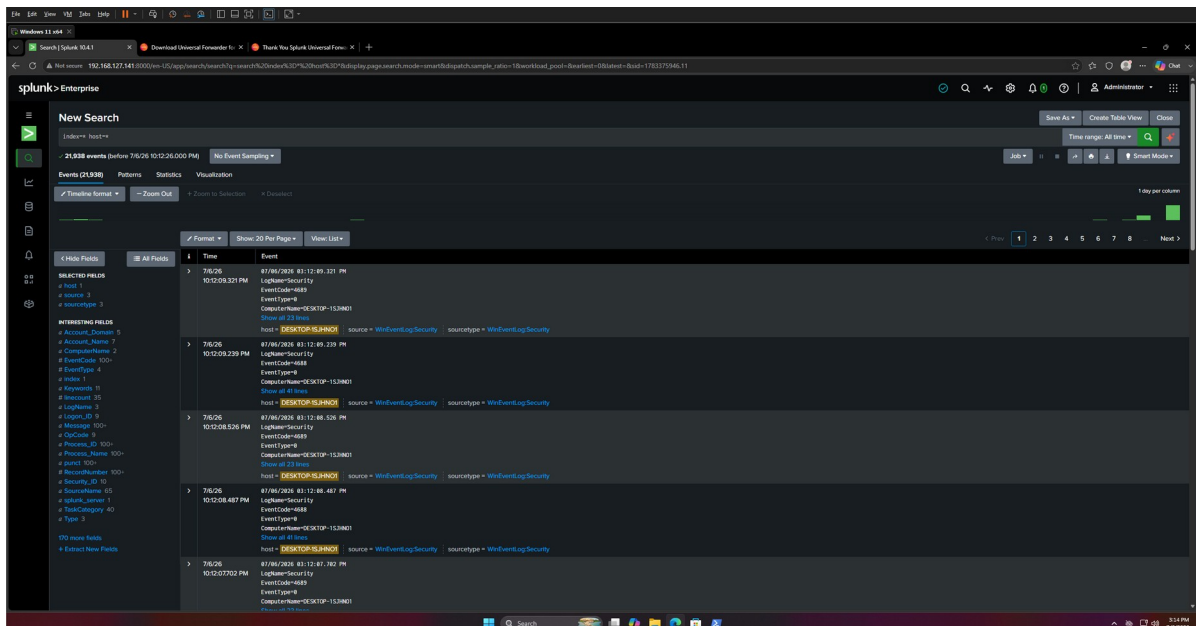
Status Name          DisplayName
-----
Running Sysmon64 Sysmon64
```

1Get-Service_Sysmon.png

3.2 Splunk Universal Forwarder

I installed the Universal Forwarder on Windows and pointed it at the Ubuntu Splunk receiver on port 9997. I enabled receiving on the Splunk side via **Settings → Forwarding and receiving → Configure receiving**.

First check showed Windows Security event codes 4688/4689 (process creation/termination) and 4703 (token adjustment) flowing in I had over 21,000 events indexed from the endpoint on the first run:



4Splunk_working_log_Ingestion.png

3.3 Sysmon Ingestion Gap and Fix

Sysmon itself was generating events locally (I confirmed active logging with Get-WinEvent), but nothing was showing up in Splunk. When I dug into the forwarder log I found the actual error:

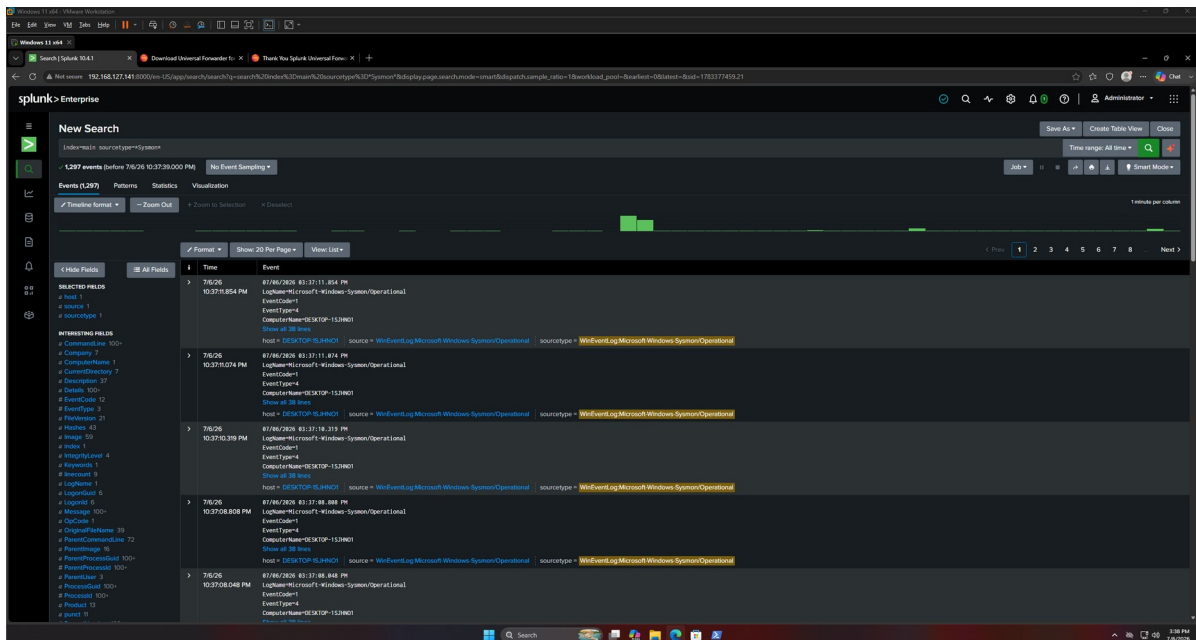
WinEventChannel::init: Init failed, unable to subscribe... errorCode=5

Error 5 is Windows generic Access Denied. The Universal Forwarder runs by default under a low-privilege virtual service account (NT SERVICE\SplunkForwarder), and that account didn't have read access to Sysmon's non-standard event channel (Microsoft-Windows-Sysmon/Operational).

I went with the least-privilege fix adding the service account to the built-in Event Log Readers group rather than elevating the whole forwarder to Local System:

net localgroup "Event Log Readers" "NT SERVICE\SplunkForwarder" /add Restart-Service SplunkForwarder

That gave the forwarder just enough to read the Sysmon channel without granting broader system rights. Sysmon events started flowing in 1,297 events with full process command line, hash, and parent-process metadata visible in Splunk:



5Working_Sysmon_log_and_ingestion.png

4. Splunk Boot Persistence

To avoid manually restarting Splunk after every VM reboot, I registered it as a boot-start service:

```
sudo /opt/splunk/bin/splunk enable boot-start -user root
```

That installs a legacy init.d-style service (I verified with `service splunk status`) so Splunk comes up on its own from here on. Elasticsearch and Kibana I set up the same way via `systemctl enable`, and both survived a full VM reboot without any manual intervention.

5. Elastic Stack Installation (Ubuntu)

I installed Elasticsearch and Kibana natively via Elastic's official APT repository, a deliberate choice over Docker to keep the stack easier to reason about and avoid stacking a container-troubleshooting layer on top of a service-troubleshooting layer.

- I explicitly capped Elasticsearch's JVM heap (`-Xms2g -Xmx2g`) so it could coexist with Splunk's memory footprint on the same VM.
- Security auto-configuration (TLS, authentication) was on by default during install and generated a superuser password for the elastic account.

The generated password prints once during install I grabbed it from the terminal before it scrolled off:

```

Selecting previously unselected package elasticsearch.
(Reading database... 196205 files and directories currently installed.)
Preparing to unpack ./elasticsearch-9.4.3_amd64.deb...
Creating elasticsearch group... OK
Creating elasticsearch user... OK
Unpacking elasticsearch (9.4.3)...
Setting up elasticsearch (9.4.3)...
----- Security autoconfiguration information -----

Authentication and authorization are enabled.
TLS for the transport and HTTP layers is enabled and configured.

The generated password for the elastic built-in superuser is : 

If this node should join an existing cluster, you can reconfigure this with
'/usr/share/elasticsearch/bin/elasticsearch-reconfigure-node --enrollment-token <token-here>'
after creating an enrollment token on your existing cluster.

You can complete the following actions at any time:

Reset the password of the elastic built-in superuser with
'/usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic'.

Generate an enrollment token for Kibana instances with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana'.

Generate an enrollment token for Elasticsearch nodes with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s node'.

-----
### NOT starting on installation, please execute the following statements to configure elasticsearch service to start automatically using systemd
sudo systemctl daemon-reload
sudo systemctl enable elasticsearch.service
### You can start elasticsearch service by executing
sudo systemctl start elasticsearch.service
Processing triggers for procps (2:4.0.4-9ubuntu1)...
Processing triggers for systemd (259.5-0ubuntu3)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

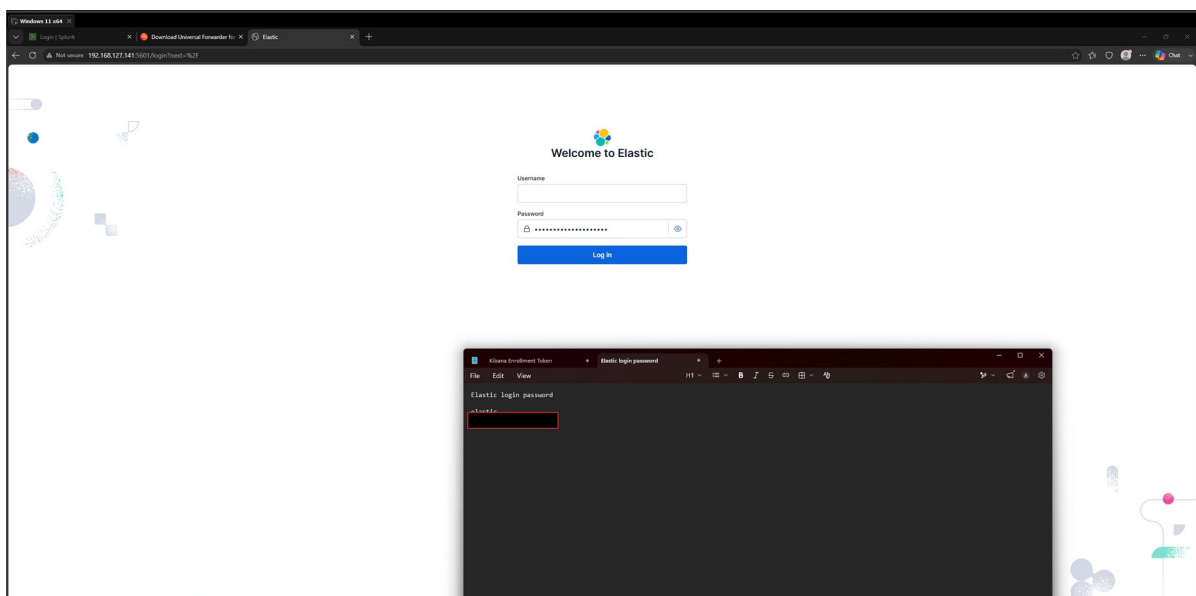
No VM guests are running outdated hypervisor (qemu) binaries on this host.
akshaybuntuserver@akshaybuntuserver:/opt/splunk/bin$ _

```

6ElasticSearchGeneratedPassword.png

Networking issue: Kibana bound only to 127.0.0.1 out of the box, so the Windows VM browser couldn't reach it same class of issue as the earlier Splunk web UI binding. I fixed it by uncommenting `server.host: "0.0.0.0"` in `/etc/kibana/kibana.yml`.

I enrolled Kibana against Elasticsearch using an enrollment token (had to regenerate it once the first one expired mid-troubleshooting) plus a one-time verification code from Kibana's own logs. Login worked with the saved elastic credentials:



7Elastic_username_and_password.png

6. Elastic Agent Deployment (Windows VM)

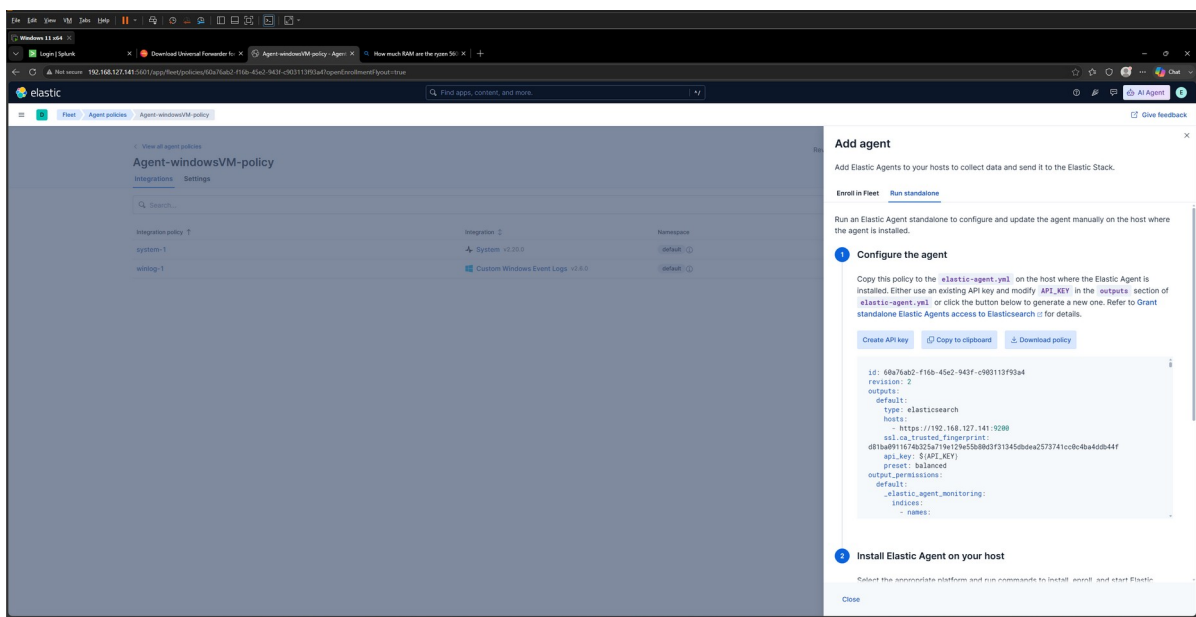
Rather than duplicate what the Universal Forwarder was already doing for Splunk, I deployed Elastic's own lightweight shipper Elastic Agent on the Windows VM to push Windows Event Logs (including the Sysmon channel) into Elasticsearch in parallel with Splunk. Two independent SIEMs receiving the same source, which is what the brief called for.

- I configured a **Custom Windows Event Logs** integration in Kibana's Fleet UI, targeting the Microsoft-Windows-Sysmon/Operational channel.
- I deliberately skipped Fleet Server enrollment in favour of **standalone mode** for a single-agent lab there's no point standing up a full Fleet Server. The standalone policy YAML gets generated in Kibana with the API key embedded and I installed it directly on the endpoint.

First attempt threw an error: "Agent binary source needs encrypted saved object api key to be set." That was Kibana asking for its encryption keys, which weren't generated on install. I fixed it with:

sudo /usr/share/kibana/bin/kibana-encryption-keys generate

and pasted the output into kibana.yml. Standalone policy screen with the generated YAML ready to download:



8Elastic_Standalone_Agent_configuration.png

On the Windows side, I downloaded the policy, extracted the agent archive, and ran `.\elastic-agent.exe install`. I chose not to enroll in Fleet when prompted (standalone), and the service came up cleanly:

```

Windows 11 x64 - VMware Workstation
File Edit View VM Tabz Help
akshaybuntuserver@akshaybuntuserver: ~
8 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Jul 7 13:17:01 2026 from 192.168.127.134
akshaybuntuserver@akshaybuntuserver:~$

Administrator: Windows PowerShell
Mode                LastWriteTime         Length Name
-----
d-----          7/7/2026  11:46 AM             elastic-agent-9.4.3-windows-x86_64

PS C:\Users\akshay\Downloads>
PS C:\Users\akshay\Downloads> Expand-Archive -Path "elastic-agent.zip" -DestinationPath "." -Force
PS C:\Users\akshay\Downloads> dir elastic-agent-9.4.3*

    Directory: C:\Users\akshay\Downloads

Mode                LastWriteTime         Length Name
-----
d-----          7/7/2026  11:57 AM             elastic-agent-9.4.3-windows-x86_64

PS C:\Users\akshay\Downloads> cd elastic-agent-9.4.3-windows-x86_64
PS C:\Users\akshay\Downloads\elastic-agent-9.4.3-windows-x86_64> Copy-Item "$env:USERPROFILE\Downloads\elastic-agent.yml" -Destination ".\elastic-agent.yml" -Force
PS C:\Users\akshay\Downloads\elastic-agent-9.4.3-windows-x86_64> .\elastic-agent.exe install
Elastic Agent will be installed at C:\Program Files\Elastic\Agent and will run as a service. Do you want to continue? [Y/n]:Y
Do you want to enroll this Agent into Fleet? [Y/n]:n
[ ] Service Started [4s] Elastic Agent successfully installed, starting enrollment.
[ ] Done [4s]
Elastic Agent has been successfully installed.
PS C:\Users\akshay\Downloads\elastic-agent-9.4.3-windows-x86_64> Get-Service "Elastic Agent"

Status  Name      DisplayName
-----
Running Elastic Agent    Elastic Agent

PS C:\Users\akshay\Downloads\elastic-agent-9.4.3-windows-x86_64>

```

9Elastic_Agent_running.png

I verified end-to-end via Elasticsearch's `_cat/indices` API the `.ds-logs-winlog.winlog-default` index alone held **5,215 documents**, alongside system metrics indices (CPU, memory, network, filesystem, process) that the base agent policy collects automatically:

```

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is not enabled.

Updates can be applied immediately.
Of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Installable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Jul  7 13:17:01 2026 from 192.168.127.134
kshaybuntuserver@kshaybuntuserver:~$ curl -k -u [REDACTED] https://localhost:9200/_cat/indices?v
health status index      uuid      pri rep docs.count docs.deleted store.size pri.store.
size dataset.size
green open      .ds-.workflows-events-2026.07.07-000001      golK5NTlQM2RZzB2q8RGTQ      1  0      0      0      249b
49b      249b
yellow open      .ds-metrics-elastic_agent.elastic_agent-default-2026.07.07-000001      387tFLtRREyxkiCFsFOGcQ      1  1      17      0      192kb      1
2kb      192kb
yellow open      .ds-logs-system.application-default-2026.07.07-000001      1sIoN20oTwykmmNFGU4jFQ      1  1      90      0      93.5kb      93
5kb      93.5kb
yellow open      .ds-metrics-system.filesystem-default-2026.07.07-000001      HNSiWAbsQcGmgcz7H-n9w      1  1      3      0      52kb
2kb      52kb
yellow open      .ds-metrics-system.diskio-default-2026.07.07-000001      wHuSI1QJRKxR0BnkrGfrg      1  1      27      0      159.5kb      159
5kb      159.5kb
yellow open      .ds-metrics-system.fsstat-default-2026.07.07-000001      5REqGn395yua0FYl9XAGHA      1  1      3      0      48.2kb      48
2kb      48.2kb
yellow open      .ds-metrics-elastic_agent.filebeat_input-default-2026.07.07-000001      5vk6I1auRZCZhilHASO_0A      1  1      15      0      239.7kb      239
7kb      239.7kb
yellow open      .ds-logs-system.system-default-2026.07.07-000001      4cUGL3qV526Gigle5xrfjQ      1  1      247      0      394.8kb      394
8kb      394.8kb
yellow open      .ds-logs-system.security-default-2026.07.07-000001      IlCgcV5EQ8653wj4-2nsUw      1  1      19093      0      5.5mb      5
5mb      5.5mb
yellow open      .ds-logs-elastic_agent.metricbeat-default-2026.07.07-000001      uQywy-Q8TAqgFsTO-PH_ig      1  1      23      0      87.9kb      87
9kb      87.9kb
yellow open      .ds-metrics-elastic_agent.metricbeat-default-2026.07.07-000001      dgwzhgvbQKiiwRcHLKtg9Q      1  1      9      0      95.4kb      95
4kb      95.4kb
yellow open      .ds-metrics-system.network-default-2026.07.07-000001      6prgW0yeS0m0jgNaM_ePkg      1  1      41      0      164.4kb      164
4kb      164.4kb
yellow open      .ds-metrics-system.cpu-default-2026.07.07-000001      805krOY1RE-5F5-Bxx04qW      1  1      14      0      127.4kb      127
4kb      127.4kb
yellow open      .ds-metrics-system.socket_summary-default-2026.07.07-000001      abuydTLMRZyYUdYEoduInA      1  1      14      0      131.6kb      131
6kb      131.6kb
yellow open      .ds-metrics-system.uptime-default-2026.07.07-000001      HKetF0myT1q5USZRsfaltw      1  1      14      0      108.2kb      108
2kb      108.2kb
yellow open      .ds-metrics-elastic_agent.filebeat-default-2026.07.07-000001      Lc23V3weRkyMY5RilrJW8A      1  1      5      0      108.8kb      108
8kb      108.8kb
yellow open      .ds-logs-winlog.winlog-default-2026.07.07-000001      KbV3lj2UTfSPaARicZpNnQ      1  1      5215      0      2.1mb      2
1mb      2.1mb
green open      .internal.alerts-security.alerts-default-000001      Bp3F2oMptJ1qQnY24mXQQA      1  0      0      0      249b
49b      249b
yellow open      .ds-metrics-system.memory-default-2026.07.07-000001      J1aCMFzWTTUGVuuZ0xqGQ      1  1      14      0      141.6kb      141
6kb      141.6kb
yellow open      .ds-metrics-system.process.summary-default-2026.07.07-000001      2B7bPZmNTICMEju_Me3EdQ      1  1      14      0      111.1kb      111
1kb      111.1kb
yellow open      .ds-logs-elastic_agent.filebeat-default-2026.07.07-000001      N0fRL9zOTMeAFNfPm3hiXQ      1  1      43      0      27.3kb      27
3kb      27.3kb
yellow open      .ds-metrics-system.process-default-2026.07.07-000001      Axk0FK2FQ107j8uXiAeakg      1  1      125      0      459.4kb      459
4kb      459.4kb
yellow open      .ds-logs-elastic_agent-default-2026.07.07-000001      sTCRCy_jTYONiRhk0ILfg      1  1      82      0      61.1kb      61
1kb      61.1kb
kshaybuntuserver@kshaybuntuserver:~$

```

10Splunk_and_Elastic_for_Log_collection.png

7. LogAI AI-Driven Log Anomaly Detection

I chose LogAI (Salesforce's open-source log anomaly detection library) to fulfil the "AI-driven anomaly detection" requirement in the brief. I ran it against a 500-row CSV export of Windows Security events I pulled from Splunk.

Environment challenge: Ubuntu 26.04's default Python (3.14) is too new for LogAI's dependency chain specifically gensim, whose C extensions won't compile against newer CPython internals. I fixed it by installing Miniconda and standing up an isolated Python 3.10 environment for LogAI alone, so system Python stayed untouched.

The pipeline I built: CSV > Preprocessor (strips variable tokens) > LogParser (Drain algorithm, template extraction) > LogVectorizer (TF-IDF) > FeatureExtractor (time-bucketed aggregation) > AnomalyDetector (One-Class SVM).

I hit three iterative bugs. I fixed each one by actually inspecting the object structures at the failure point instead of guessing:

1. I passed the full parsed-log DataFrame into feature extraction instead of the specific `parsed_logline` column.
2. I left out the vectorization step entirely text has to be converted to numeric vectors before feature aggregation. Missed it on the first pass.
3. The resulting feature vector still had a non-numeric timestamp column, which scikit-learn's SVM refused to touch. Fix: I dropped the timestamp column immediately before `.fit()` / `.predict()`, but kept it in a side variable so I could still label which time buckets got flagged.

Time-bucket refinement: my initial `group_by_time="15min"` setting collapsed the 10-minute sample span into just 2 buckets nowhere near enough for meaningful anomaly detection. After I printed the actual timestamp range (`min=20:22:09`, `max=20:32:29`), I dropped the bucket size to `"1min"`, giving me 11 usable buckets.

Nu parameter tuning: the One-Class SVM's default nu (0.5) flagged about 45% of buckets as anomalous (5 out of 11) that's the model's sensitivity setting, not a real threat signal. Reading LogAI's `OneClassSVMParams` config class, I set nu=0.1 (target ~10% anomalies). That meant I had to pass a proper `OneClassSVMParams` object rather than a plain dict. LogAI's factory pattern enforces type-checked config objects per algorithm.

Final result: with $\nu=0.1$, the detector flagged one time bucket as anomalous the 20:30:00 UTC window on July 7, 2026. That's a defensible baseline result for a clean lab environment with no active attack traffic:

```
[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word.py python -c "from logapi.analysis.anomaly_detector import AnomalyDetectionConfig; help(AnomalyDetectionConfig)"

[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word $id_config - AnomalyDetectionConfig
[logali] syntax error near unexpected token '(''
[logali] syntax error near unexpected token '$ id_config - AnomalyDetectionConfig('
[logali] syntax error near expected token ','
[logali] custom not found
[logali] syntax error near unexpected token ')'
[logali] syntax error near unexpected token './logali_word.py python -c "'
[logali] from logapi.analysis.anomaly_detector import AnomalyDetectionConfig
[logali] from anomaly_detection.config import algop_name='one_class_svm', algo_params={'nu': 0.1})
[logali] print(c)
[logali] AnomalyDetectionConfig(algo_name='one_class_svm', algo_params={'nu': 0.1}, custom_params=None)
[logali] [logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word $name logapi_run.py
[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word python logapi_run.py
Loading CSV...
Loaded SMO run
SMO rank have valid timestamps
TimeSpan spans 8 days 00:16:29.770000 (min=2026-07-07 20:22:09.105000+00:00, max=2026-07-07 20:22:10.440000+00:00)
Parsing log templates...
Vectorizing parsed log templates (TF-IDF)...
Extracting features (bucket size=360)...
Number of time buckets: 11
Running anomaly detection (One-Class SVM)...
Traceback (most recent call last):
File "/home/akshay/huntserver/logapi/work/logapi_run.py", line 58, in <module>
    file = open(os.path.join(logapi_dir, 'anomalydetector.log'), mode='w')
FileNotFoundError: [Errno 2] No such file or directory: '/home/akshay/huntserver/dataset/logapi/lib/python3.10/site-packages/logapi/analysis/anomaly_detector.py', line 44, in _init_
File "/home/akshay/huntserver/factory/get_algorithm.py", line 11, in get_algorithm
File "/home/akshay/huntserver/algorithms/emvs/logapi/lib/python3.10/site-packages/logapi/algorithms/factory.py", line 117, in get_algorithm
AttributeError: Config must be an instance of OneClassSVMParams.
File "<string>", line 1
from logapi.analysis.anomaly_detector.algo_one_class_svm import OneClassSVMParams; help(OneClassSVMParams).
SyntaxError: invalid syntax
[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word.py python -c "from logapi.algorithms.anomaly_detection.algo_one_class_svm import OneClassSVMParams; help(OneClassSVMParams)"

[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word $name logapi_run.py
[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word python logapi_run.py
Loading CSV...
Loaded SMO run
SMO rank have valid timestamps
TimeSpan spans 8 days 00:16:29.770000 (min=2026-07-07 20:22:09.105000+00:00, max=2026-07-07 20:22:10.440000+00:00)
Parsing log templates...
Vectorizing parsed log templates (TF-IDF)...
Extracting features (bucket size=360)...
Number of time buckets: 11
Running anomaly detection (One-class SVM)...

=== ANOMALY DETECTION RESULTS ===
anom_score_traind
   anom      1     False
   score     1     False
   count     1     False
   name      1     False
Name: count, dtype: int64

1 time-bucket(s) flagged as anomalous!
TimeSpan
8 2026-07-07 20:16:00+00:00

Done.
[logali] akshay@ubuntu-server:/akshay/huntserver$ ./logali_word $
```

12Log-ai script output.png

```

-bash: syntax error near unexpected token `~'\nDone.'"
(logai) akshaybuntuser@akshaybuntuser:~/logai-work$ nano logai_run.py
(logai) akshaybuntuser@akshaybuntuser:~/logai-work$ python logai_run.py
Loading CSV...
Loaded 500 rows
500 rows have valid timestamps
Timestamp span: 0 days 00:10:20.275000 (min=2026-07-07 20:22:09.165000+00:00, max=2026-07-07 20:32:29.440000+00:00)
Parsing log templates...
Vectorizing parsed log templates (TF-IDF)...
Extracting features (bucket size=1min)...
Number of time buckets: 11
Running anomaly detection (One-Class SVM)...

=== ANOMALY DETECTION RESULTS ===
anom_score  trainval
0           1      False
1           1      False
2           1      False
3           1      False
4           1      False
anom_score
1           6
-1          5
Name: count, dtype: int64

5 time-bucket(s) flagged as anomalous:
timestamp
5 2026-07-07 20:27:00+00:00
7 2026-07-07 20:29:00+00:00
8 2026-07-07 20:30:00+00:00
9 2026-07-07 20:31:00+00:00
10 2026-07-07 20:32:00+00:00

Done.
(logai) akshaybuntuser@akshaybuntuser:~/logai-work$ client_loop: send disconnect: Connection reset
PS C:\Users\akshay\Downloads>

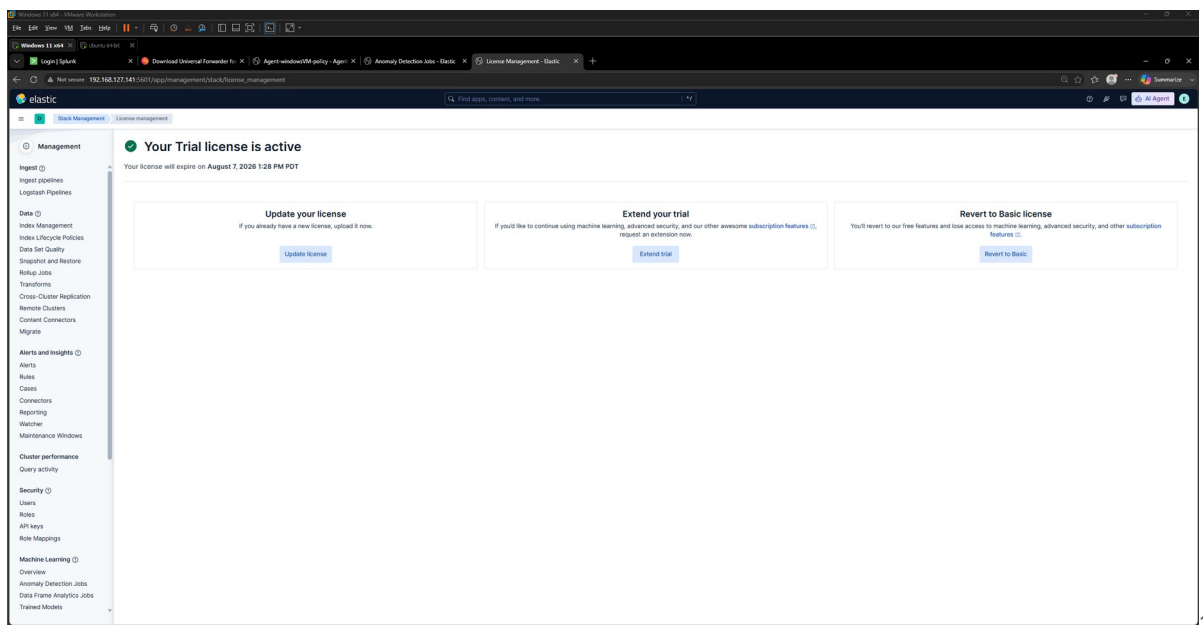
```

11_logai_anomaly_results.png

8. Task 2 - AI-Driven Detection and Threat Intelligence

8.1 Elastic ML Trial License

Elastic Machine Learning needs a trial or paid license it's not in the default Basic tier. I activated a 30-day trial through **Stack Management to License Management** in Kibana, which unlocked anomaly detection job creation. Trial expiry: August 7, 2026:



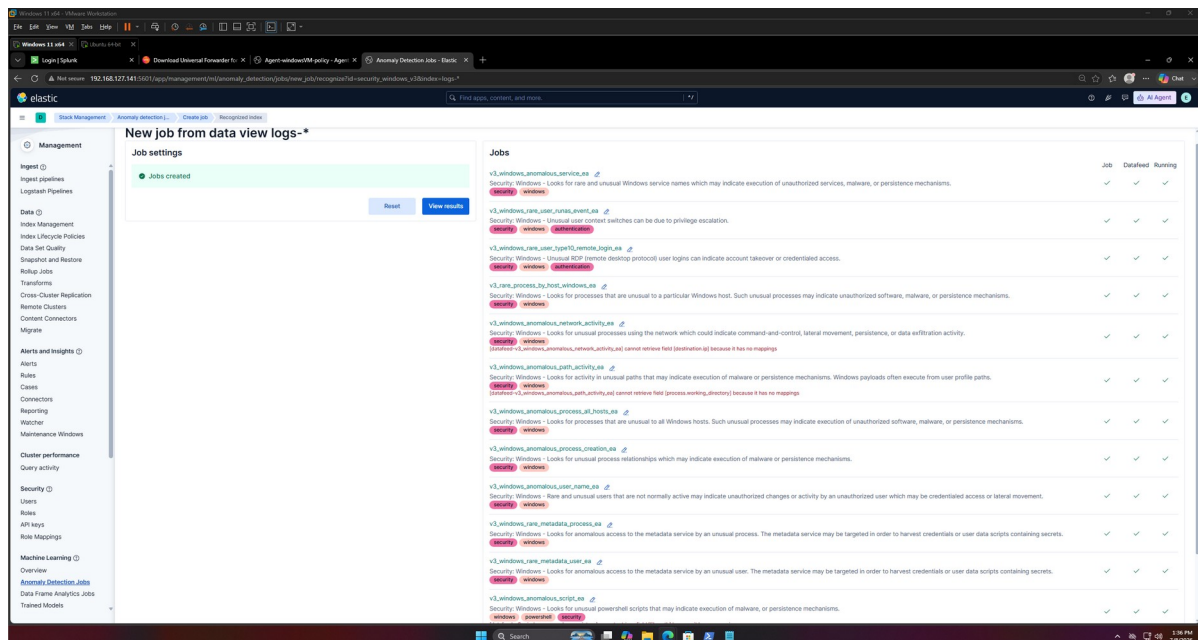
Elastic_Trial_License.png

8.2 Elastic ML Anomaly Detection Jobs

Using the trial, I created the prebuilt "Security: Windows" anomaly detection jobs against the logs-* data view. These are engineered for host-based threat hunting on ECS-compatible logs. Coverage includes:

- Rare / unusual Windows service names (potential malware, persistence)
- Unusual user context switches (potential privilege escalation)
- Unusual RDP logins (potential account takeover)
- Rare processes by host (potential unauthorized software)
- Anomalous network activity (potential C2 or lateral movement)
- Anomalous path activity (execution from user profile paths classic malware pattern)
- Anomalous process creation relationships
- Anomalous user activity (potential unauthorized access)

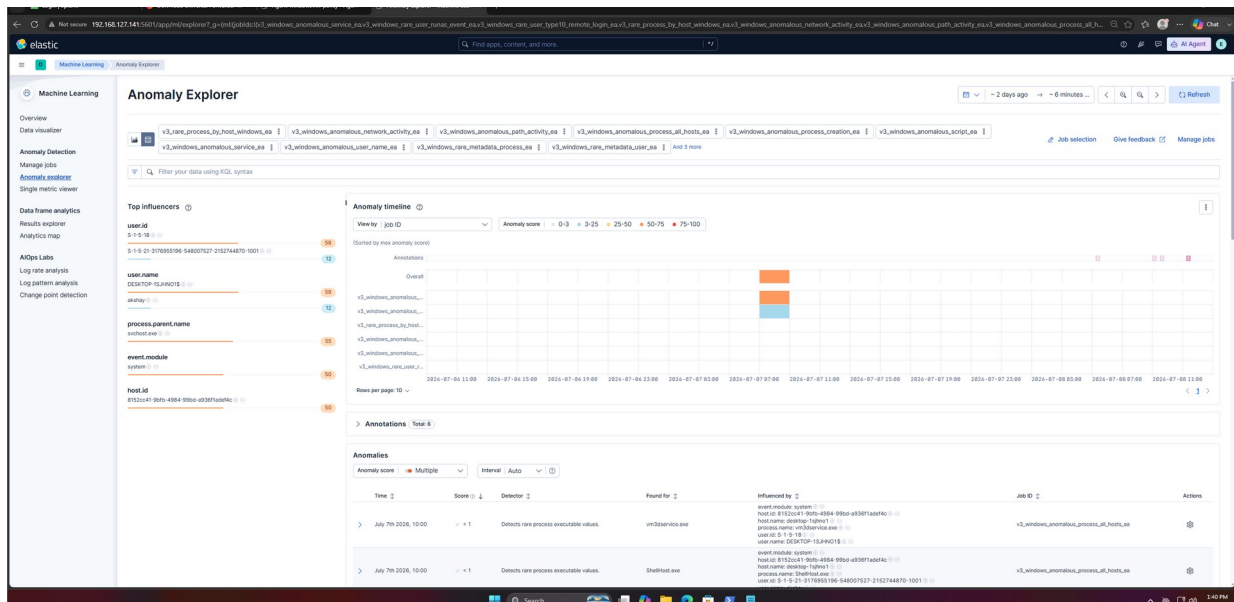
All jobs came up cleanly with datafeeds started immediately, running against the full logs-* dataset:



2Elastic_ML_Jobs_created.png

Two jobs threw field-mapping warnings: `destination.ip` missing on the network-activity job, and `process.working_directory` missing on the path-activity job. These are ECS fields the current Elastic Agent config doesn't populate. I carried them forward as a finding into the gap analysis rather than paper over them.

Anomaly Explorer confirmed real detection output I could see anomalies identified in the July 7 window with top influencers including specific user IDs, the `svchost.exe` parent process, and the system event module. Scores were low (below 1), which is expected in a clean lab with no attack traffic yet. The scores would climb once real attack activity was fed in, which happens in Task 3:



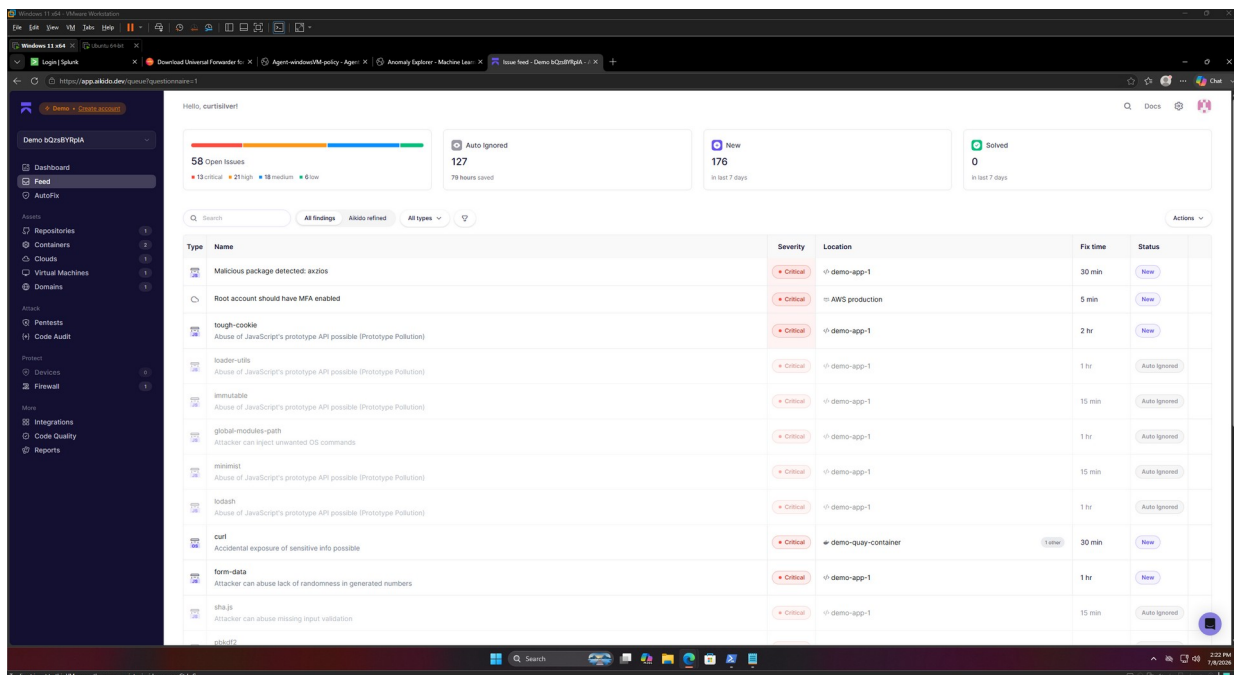
3Elastic_ML_Anomaly_Detected.png

8.3 Aikido AI-Driven Application Security

I connected Aikido via its demo workspace (GitHub auth) to evaluate its AI-driven detection at the application security layer, as called for in the brief. Aikido scans source code repos, containers, and cloud configs for things like malicious packages, exposed secrets, prototype pollution, and infrastructure misconfigurations.

The demo workspace surfaced 58 open issues (13 critical, 21 high, 18 medium, 6 low) across a sample application findings included the malicious npm package axzios, missing MFA on cloud root accounts, and multiple JavaScript prototype-pollution vulns.

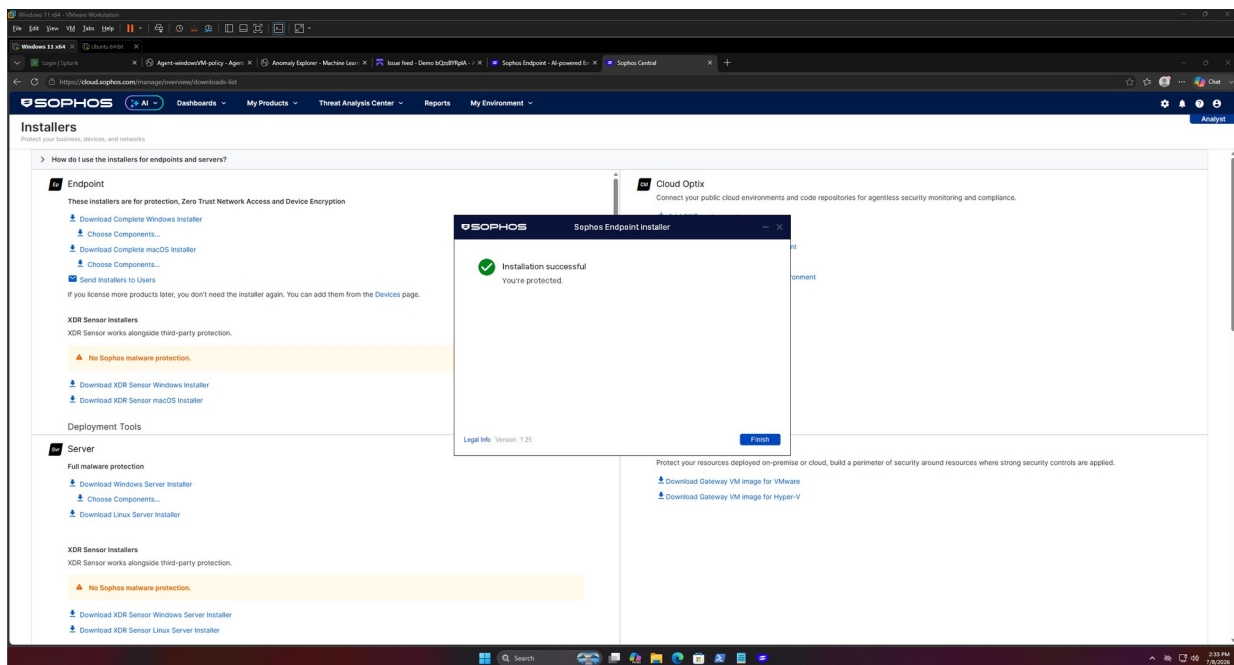
I want to be clear about where Aikido fits: it operates at the code and configuration layer, complementary to but distinct from the log-based behavioural anomaly detection that LogAI and Elastic ML do in the SIEM. Aikido looks at what could be exploited before it runs; the SIEM tools look at what's happening after it runs. Two different visibility layers:



4Aikido_Demo_Dashboard.png

8.4 Sophos Intercept X : Endpoint Protection with AI

I deployed Sophos Intercept X on the Windows 11 endpoint as the EDR / XDR layer specified in the brief. I downloaded the Windows installer from the Sophos Central management console, ran it on the endpoint, and the installer confirmed successful:



5Sophos_Endpoint_Installed.png

Sophos Central then showed the Windows VM as online, tamper protection enabled, XDR agent active, encryption and DNS protection all green. Deep learning-based malware detection, ransomware protection, and exploit prevention all run natively on the endpoint from here.

8.5 OpenCTI Threat Intelligence Platform

I deployed OpenCTI on the Ubuntu SIEM VM via Docker Compose. Docker Engine (v29.1.3) and Docker Compose (v2.40.3) went in via apt, then I pulled OpenCTI's multi-container config from github.com/OpenCTI-Platform/docker.

The environment file needed several credentials and tokens up front:

- Admin credentials (email + password)
- API token, I generated via uuidgen
- Platform registration token openssl rand -hex 32
- Encryption key openssl rand -base64 32
- Internal service passwords for MinIO, RabbitMQ, and OpenSearch

The full stack is 30 containers, OpenCTI's core platform, Elasticsearch, Redis, RabbitMQ, MinIO (object storage), PostgreSQL, plus multiple data connectors for import/export in STIX, CSV, TXT, YARA, and document formats, and a dedicated MITRE ATT&CK connector.

I ran into a Docker permissions issue on the first docker compose up -d my user account wasn't in the docker group yet. I fixed it with:

```
sudo usermod -aG docker $USER
```

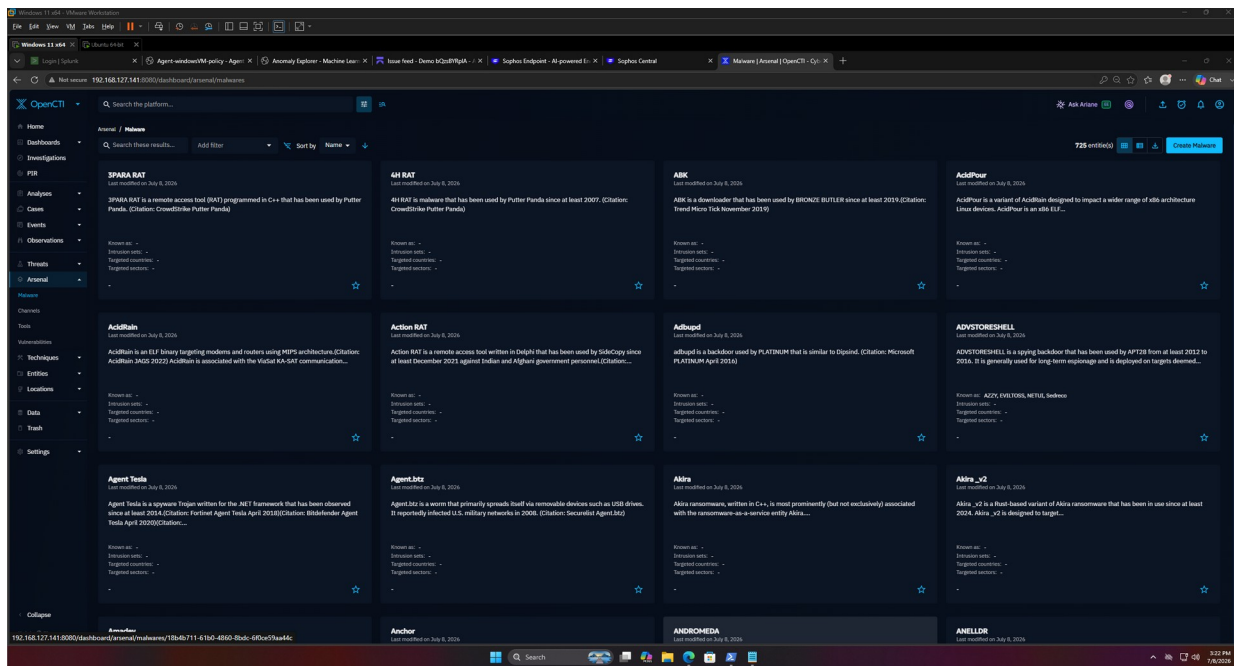
followed by a session re-login for the group membership to apply. All 30 containers came up healthy, each showing Healthy or Started:

[illegible]

6OPENCTI containers running.png

On first boot the MITRE connector auto-populated the platform with baseline threat intelligence: **725 malware entities** and **181 intrusion sets**, each with descriptions, known aliases, and metadata

sourced from the MITRE ATT&CK knowledge base. That covers both the "OpenCTI stood up" and "at least one threat intel feed connected" requirements from the brief:



7OPENCTI_threat_intel_populated.png

9. Task 3 | Threat Hunting and IOC Pivoting

9.1 Attack Traffic Generation

I executed two attack techniques from Kali (192.168.127.131) against the Windows endpoint (192.168.127.134): a network recon scan and a credential attack over SMB.

Network reconnaissance Nmap SYN scan. I ran a full-range TCP SYN scan (`sudo nmap -sS -Pn -p- 192.168.127.134`) across all 65,535 ports. It took 604 seconds and identified one open port 7680/tcp (Windows Delivery Optimization / pando-pub) with the remaining 65,534 ports filtered by Windows Firewall:

I followed up with a targeted scan against the ports I actually cared about (`sudo nmap -sS -Pn -T4 -p 80,443,445,3389,7680 192.168.127.134`). Confirmed 80, 443, 445, and 3389 as filtered, 7680 open, done in 1.9 seconds:



Credential attack : SMB failed login. After I temporarily enabled the "File and Printer Sharing" firewall rule so SMB traffic could actually reach the auth subsystem, I made a failed SMB login attempt with fabricated credentials:

Page 19 of 36

Windows rejected the auth with `NT_STATUS_LOGON_FAILURE`, which is exactly what I wanted that proves the connection reached the target and was processed by the authentication subsystem. I also tried an RDP connection with `xfreerdp` just to generate additional failed-auth telemetry:



3Kali failed login attempt.png

9.2 Enabling Firewall Visibility

This turned into one of the more important findings of the whole project: **Windows Firewall does not log blocked connections by default.**

The first hunt I ran in Splunk for the attacker's IP came back with zero results. Not because the scan didn't happen I could see it hitting the target from Kali. It was because the firewall silently dropped every packet without generating a single log entry.

I turned that on across all three profiles:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -LogBlocked True -LogFileName  
"%systemroot%\system32\LogFiles\Firewall\pfirewall.log" -LogMaxSizeKilobytes 4096
```

I verified all three profiles had `LogBlocked: True` set. Then I added the log file as a monitored input in the Universal Forwarder so blocked connections would land in Splunk:

```
.\splunk.exe add monitor "C:\Windows\System32\LogFiles\Firewall\pfirewall.log" -index main -sourcetype "WinFirewallLog"
```

Once that was in place, the raw firewall log started showing DROP entries for TCP traffic from 192.168.127.131 across a whole range of destination ports the same nmap scan I'd already run, now visible where it hadn't been before:

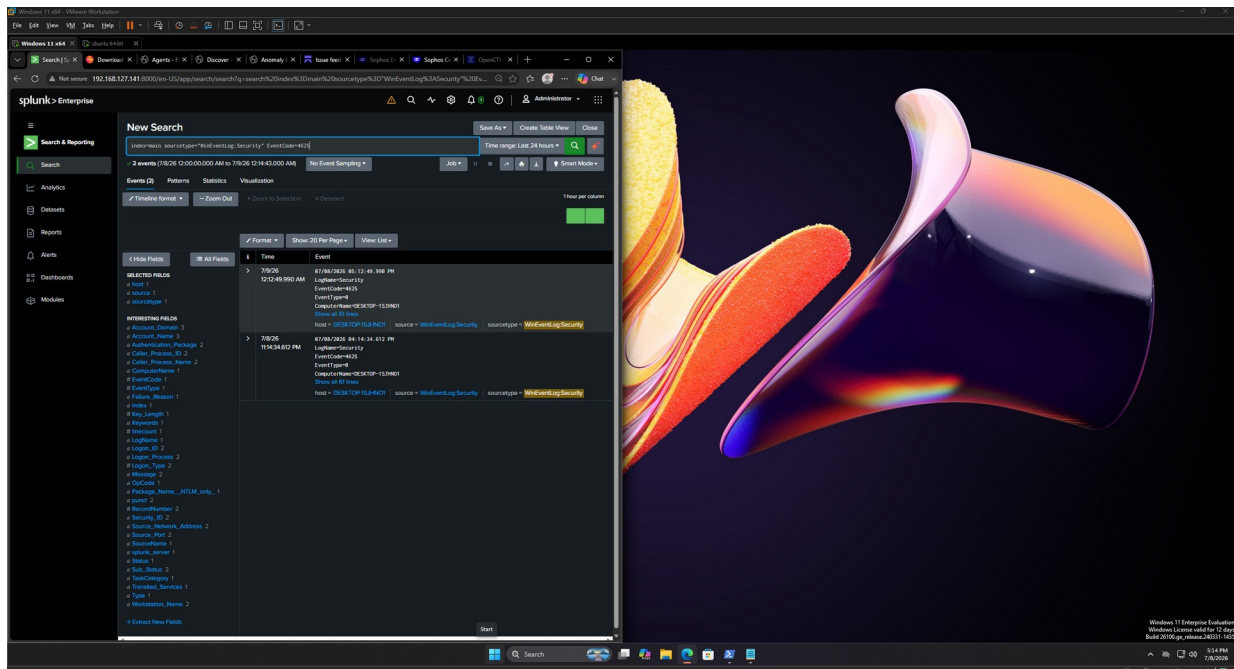
9.3 IOC Hunt Source 1: Windows Firewall Log

index=main sourcetype="WinFirewallLog" 192.168.127.131

9.4 IOC Hunt Source 2: Windows Security Event Log

index=main sourcetype="WinEventLog:Security" EventCode=4625

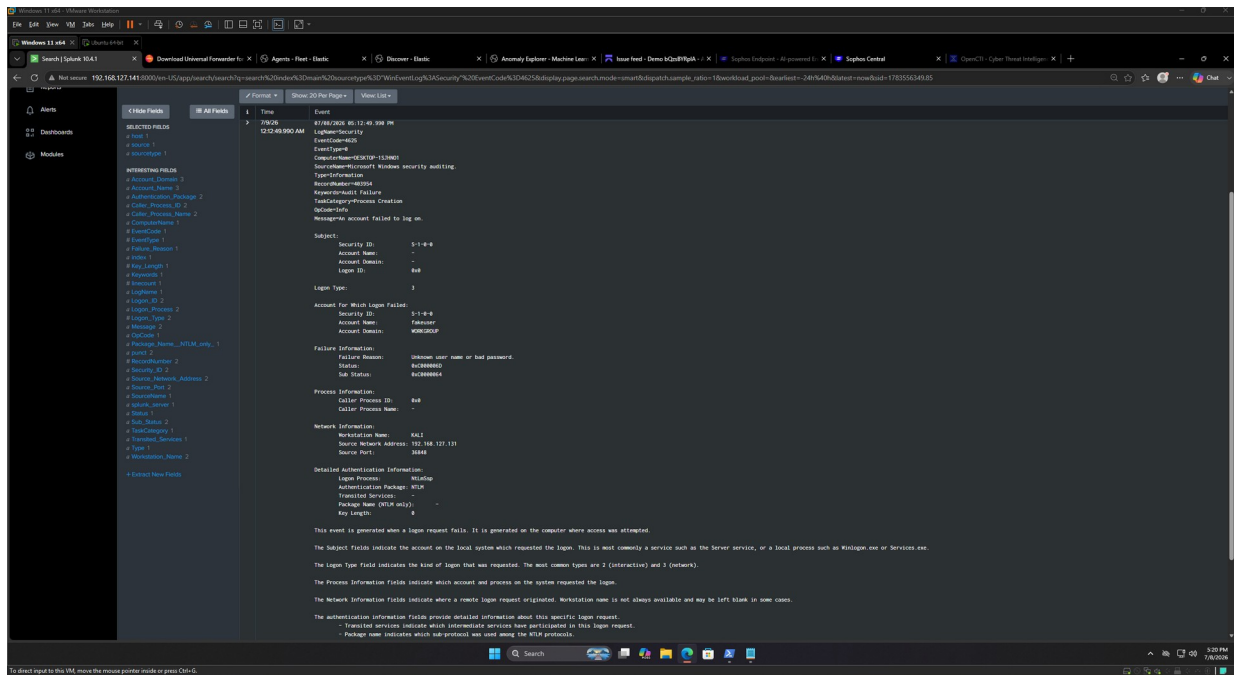
Page 21 of 36



4Splunk4625_events.png

When I expanded the event detail I nailed the cross-source correlation every field I needed to tie the network-layer telemetry to the auth-layer telemetry was in there:

- Source Network Address: 192.168.127.131 (matches the firewall DROP source)
- Workstation Name: KALI
- Account Name: fakeuser
- Failure Reason: Unknown user name or bad password
- Logon Type: 3 (network logon | SMB)
- Authentication Package: NTLM



5Splunk4625_event_Detail.png

9.5 Cross-Source Pivot Summary

The attacker IP (192.168.127.131) acted as my pivot point across two independent log sources inside Splunk:

1. Windows Firewall log (sourcetype: WinFirewallLog) network-level recon. 18 blocked TCP connection attempts across multiple ports, corresponding to my Nmap SYN scan.
2. Windows Security event log (sourcetype: WinEventLog:Security) application-level credential attack. 2 failed logon events (EventCode 4625) with fabricated credentials, corresponding to my SMB attempt.

Two data sources independently identified the same source IP, corroborating the attacker's activity across different layers of the defense stack network filtering vs authentication logging. That's exactly the value proposition of centralised log ingestion: no single log source would have given me the full picture, but the correlation across two of them makes the attack unambiguous.

9.6 Key Observations from the Hunt

- **Windows Firewall logging for blocked connections is disabled by default.** Without explicitly enabling it, my entire port scan would have been invisible to the SOC. That's a real detection gap in vanilla Windows and it goes into the gap analysis.
- **Sysmon Event ID 3 (Network Connection) captured nothing for this scan.** The SwiftOnSecurity config keeps network logging conservative to manage volume, and a SYN scan by design never completes the handshake that would trigger connection-level logging. Compounding factor with the default-off firewall log if both are silent, the recon phase is invisible.

- **The two log sources are complementary, not redundant.** Firewall drops catch blocked reconnaissance; authentication failures catch active exploitation attempts that got past the firewall. Neither alone is sufficient.
- **Port 445 (SMB) was blocked by Windows Firewall by default.** Even my failed-login attempt couldn't reach the auth layer until I temporarily allowed "File and Printer Sharing." That's worth flagging: firewall config directly determines which attack types even show up in downstream security logs. Silently dropping the traffic means silently dropping the detection signal that goes with it.

10. Task 4 | SOAR Automation, MITRE/D3FEND Mapping, and Gap Analysis

Task 4 closes the loop by adding an automated response layer on top of the detection and threat-intel work I did in Tasks 1–3. The rubric required me to: stand up Shuffle (SOAR) and TheHive (case management), build and trigger at least one automated playbook, map coverage to MITRE ATT&CK, map controls to D3FEND, and produce a gap analysis of what's still uncovered.

10.1 Shuffle Deployment

I deployed Shuffle on the Ubuntu SIEM VM via Docker Compose, cloned from the official Shuffle GitHub repo. Before starting it I stopped OpenCTI (didn't delete data volumes preserved) to free memory headroom, since the full OpenCTI stack (18 containers, roughly 6–8 GB RAM) was already stretching the host and stacking Shuffle on top would have pushed it over.

Issue: port collision on 9200. My first docker compose up -d failed with:

failed to bind host port 0.0.0.0:9200/tcp: address already in use

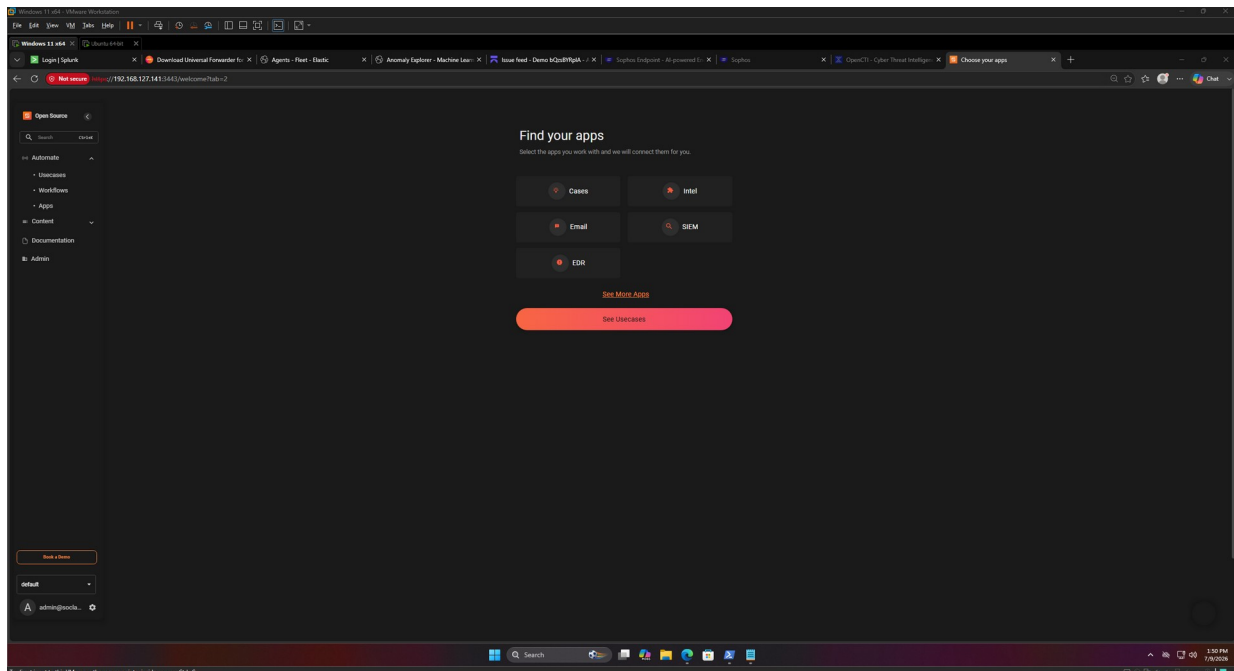
I confirmed via `sudo ss -ltnp | grep 9200` that the native Elasticsearch from Task 1 was already using 9200. Shuffle bundles its own OpenSearch, which defaults to the same port.

Fix: I remapped Shuffle's OpenSearch host-side port. Edited line 88 of `~/shuffle/docker-compose.yml`:

```
sed -i '88s/9200:9200/9201:9200/' docker-compose.yml
```

The container-internal port stayed the same, so Shuffle's backend/frontend/orborus containers still reached OpenSearch via the internal Docker network name and internal port no downstream config changes needed. After the edit, all four Shuffle containers (shuffle-opensearch, shuffle-backend, shuffle-frontend, shuffle-orborus) came up cleanly.

I accessed the Shuffle web UI at `https://192.168.127.141:3443` from the Windows VM browser (self-signed cert, click-through in a lab is fine). First launch prompted for admin account creation, then the onboarding flow ran:



1Shuffle_On-boarding.png

10.2 Playbook Design | Failed Logon Auto-Response

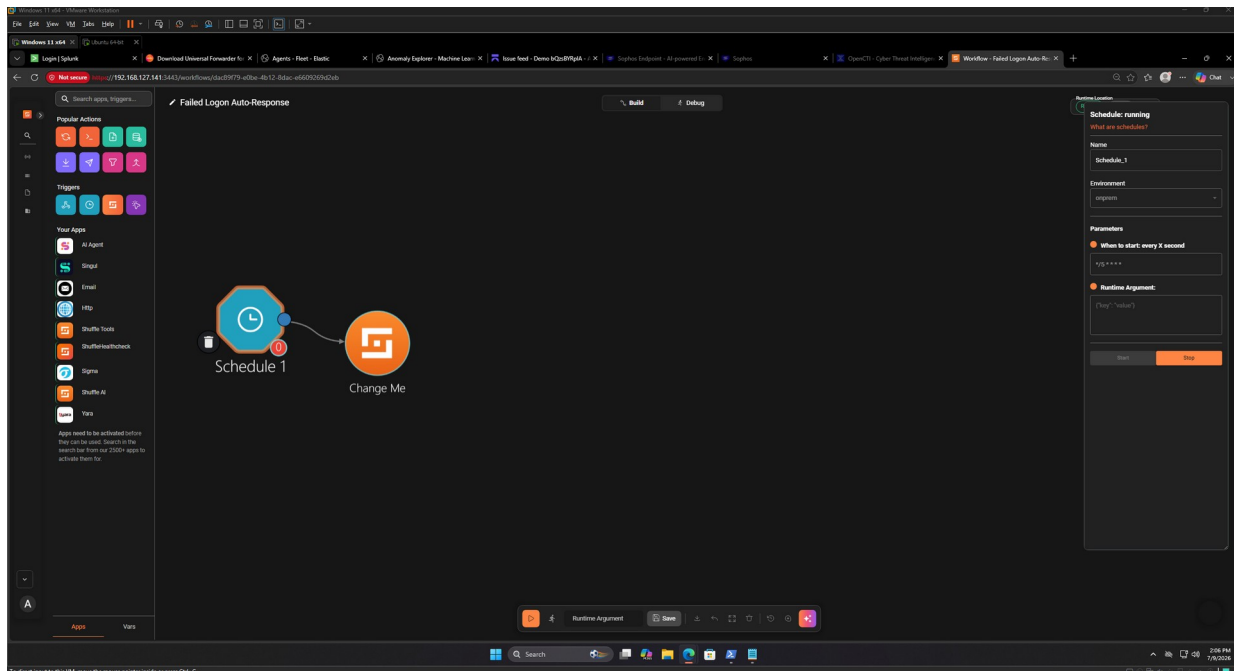
I designed the workflow *"Failed Logon Auto-Response"* to close the loop on the exact attack scenario I ran in Task 3. Same IOC I hunted manually before is now the trigger for automated case detection. That gives the whole project a coherent narrative across all four tasks instead of Task 4 living in isolation.

Three connected nodes: Schedule Trigger > Splunk Search > Filter List.

Node 1: Schedule Trigger

A cron-based trigger firing every 5 minutes (`*/5 * * * *`).

I picked scheduled polling over webhook-based triggering deliberately. A webhook would require additional Splunk alert-action configuration on the SIEM side, which mostly tests my Splunk alert setup rather than SOAR concepts. Polling isolates the automation logic to Shuffle itself and demonstrates the core SOAR pattern recurring automated detection cleanly. That's defensible for the writeup and it also happens to be closer to how a lot of real production playbooks are actually built.



2ShuffleWorkflow_Schedule_Trigger_Node.png

Node 2: Splunk Search Action

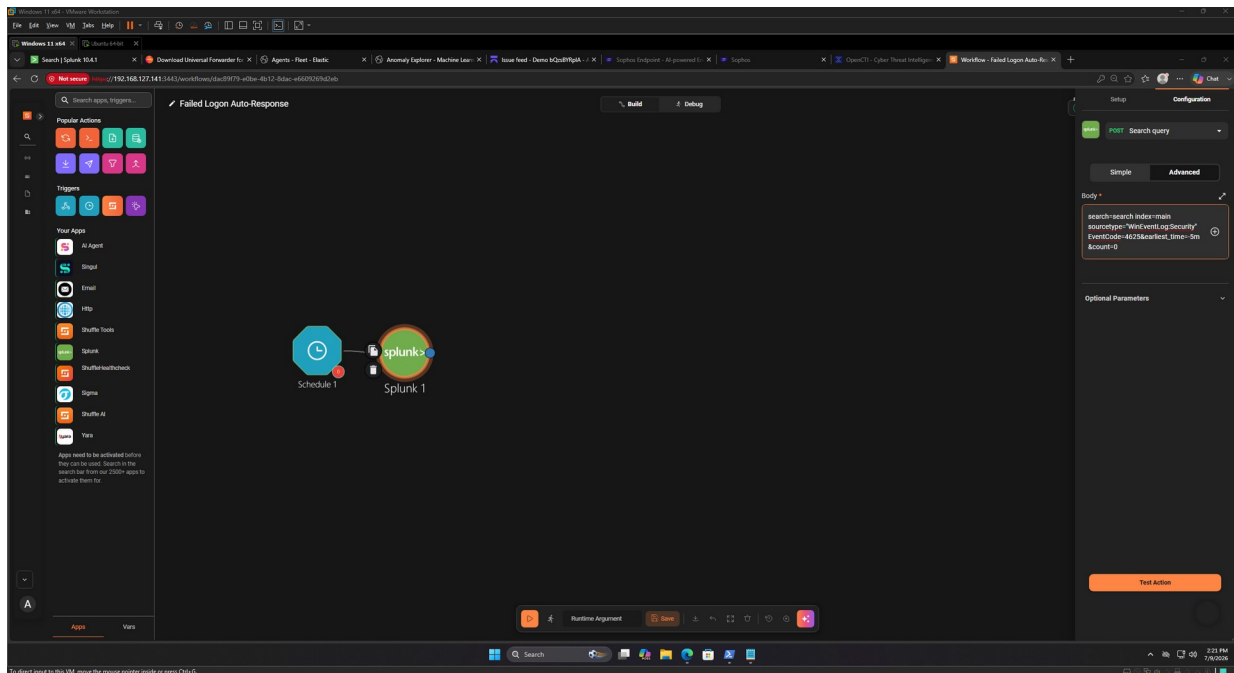
A call to Splunk's REST API on the management port (8089, distinct from the web UI on 8000). I configured auth via Shuffle's auth manager with URL <https://192.168.127.141:8089>.

My first attempt used localhost:8089 which is Shuffle's own container-internal loopback, not the host. I corrected to the VM's actual IP once I recognised the container-vs-host networking distinction. That's an honest SOAR-engineering lesson: containers have their own network namespaces and "localhost" from inside a container means the container, not the host.

I selected "Search query" from Shuffle's Splunk app dropdown (not "Notable event update" that pushes updates back to Splunk rather than reading from it). My Body field in Advanced view:

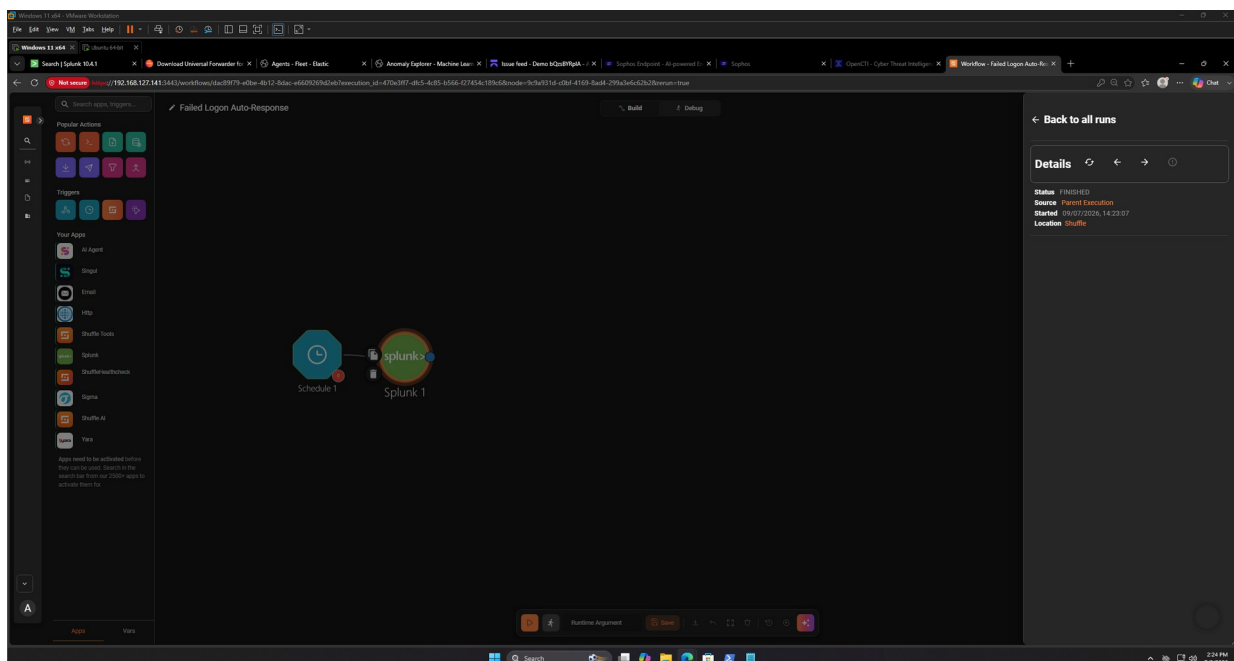
search=search index=main sourcetype="WinEventLog:Security" EventCode=4625&earliest_time=-24h&count=0

I used earliest_time=-24h during initial testing rather than the production -5m that way the existing Task 3 failed-logon events (which happened hours earlier) were still in the search window during validation.



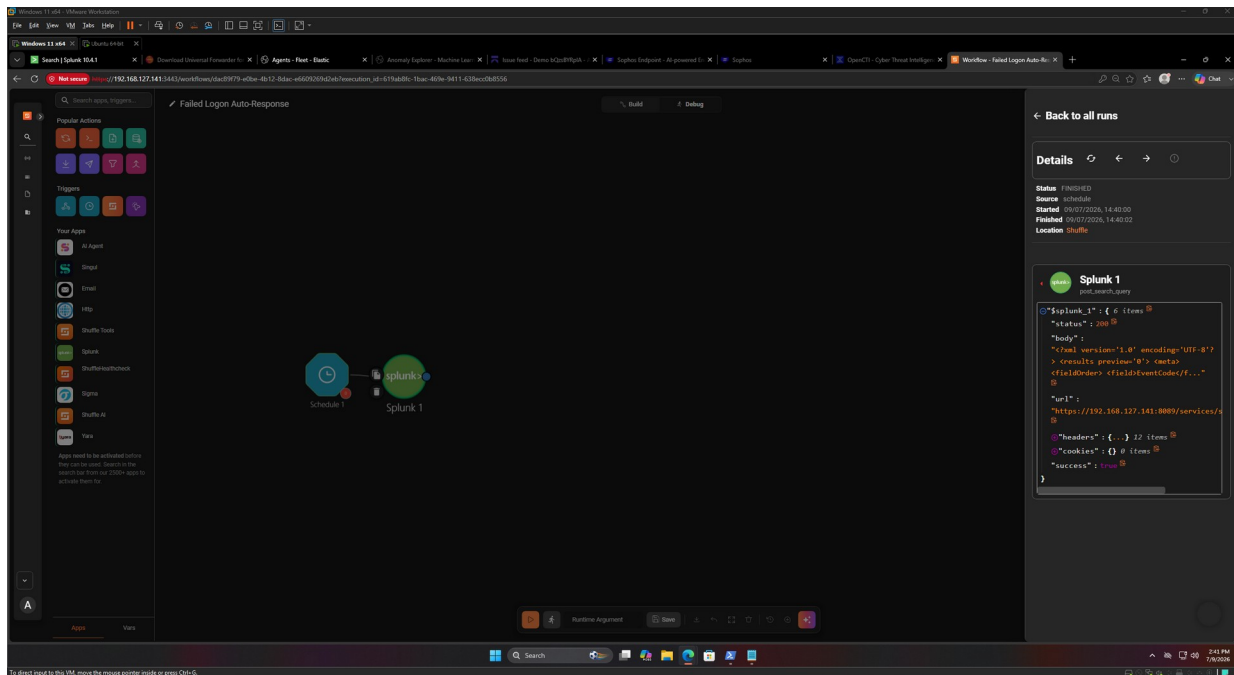
3ShuffleWorkflow_Splunk_Config.png

Test result: HTTP 200 with a valid XML response body containing <results preview='0'> and <fieldOrder> <field>EventCode</field>.... Shuffle authenticated cleanly against Splunk's REST API and got the failed-logon data back:



4Shuffle_Splunk_Search_Test.png

Scheduled execution confirming the trigger fires on its own (Source: schedule, Status: FINISHED, Splunk 1 returned status 200 with the XML body). This is the key piece of evidence for my working SOAR trigger>detection loop:



5Shuffle_Splunk_Query_Result.png

Node 3: Filter List (Shuffle Tools)

A conditional node inspecting the Splunk response body for the presence of "4625." My configuration:

- Input list: \$splunk_1.body
- Field: EventCode
- Check: Contains
- Value: 4625
- Opposite: False

This is the guard clause. The workflow only proceeds when a failed-logon EventCode is actually present in the results, not on every empty poll cycle. Without this I'd have a schedule-fires-every-5-minutes workflow that pages someone every 5 minutes whether or not there's anything to look at. With it, I have a workflow that stays quiet until it has something worth acting on.

10.3 Deployment Challenges Documented

Two challenges hit during Task 4 that I want to document straight rather than glossing over.

TheHive deployment | blocked by memory constraints

I attempted TheHive 5 with a memory-tuned Docker Compose config: Cassandra with 512 MB heap (1 GB container limit), a dedicated Elasticsearch 7.17 instance with 256 MB heap (later raised to 512 MB), and TheHive itself at 768 MB. TheHive 5 requires ES 7.x specifically, so I couldn't reuse the native ES 8 from Task 1 my dedicated ES for TheHive was mapped to host port 9202 to avoid the native ES on 9200 and Shuffle's OpenSearch on 9201.

Even with the tuning, TheHive's JVM never established a stable connection to its ES backend. It kept looping on the same retry pattern:

```
[warn] o.j.d.e.r.RestElasticSearchClient - Unable to determine Elasticsearch server version. Default to SEVEN. java.net.ConnectException: Connection refused at
org.janusgraph.diskstorage.es.rest.RestElasticSearchClient.getMajorVersion(...) [warn] o.t.s.u.Retry -
An error occurs (java.lang.IllegalArgumentException: Could not instantiate implementation:
org.janusgraph.diskstorage.es.ElasticSearchIndex), retrying (1/10) → (2/10) → (3/10) ...
```

I checked `docker inspect thehive --format='{{.State.OOMKilled}}'` it returned false, so the container wasn't being killed by the kernel. ES itself was likely too underprovisioned to accept connections reliably before TheHive's retries exhausted.

On the available RAM budget around 3.6 GiB free with Shuffle already running further tuning wasn't viable. I documented TheHive as a resource-constrained deployment, not a functional failure. That distinction matters: the tool works, but the lab host doesn't have the capacity to run it alongside everything else. I discuss this further in the gap analysis (Section 10.7).

VM suspend/resume | Docker network stability

My Ubuntu VM got suspended and resumed mid-Task-4 (I closed the laptop). After resume, the Shuffle web UI became intermittently unreachable from the Windows VM even though `docker compose ps` showed all containers Running and `sudo ss -ltnp | grep 3443` confirmed docker-proxy was still bound to 0.0.0.0:3443.

Basic ICMP between VMs was fine (ping 192.168.127.141 answered in under 1 ms), but TCP to port 3443 timed out from both the Windows VM (`curl.exe -k -I https://192.168.127.141:3443` failed after 21 seconds) and from inside Ubuntu itself when I targeted the external IP (`curl -k -I https://192.168.127.141:3443` timed out at 5 seconds). Localhost connections behaved differently "connection reset by peer" pointing at docker-proxy misbehaving post-resume.

`dmesg` also showed multiple Memory cgroup out of memory: Killed process (java) entries around the resume moment, which suggested transient memory pressure during VM state restore contributed to the instability.

Two takeaways I'm including in the writeup: production SOC infrastructure shouldn't run on suspend-prone lab VMs, and container-hosted services need to be validated after any VM state transition. Both are legitimate operational findings worth calling out.

10.4 MITRE ATT&CK Coverage Mapping

The following techniques (MITRE ATT&CK v14) are covered by the detection stack I built in Tasks 1–3. I used three status tiers: **Detected** means I explicitly captured evidence during Tasks 2 or 3; **Detectable** means the log source is present but I haven't triggered it with test traffic yet; **Partial** means some coverage exists but with known gaps in event volume or completeness.

Tactic	Technique ID	Technique Name	Detection Source in Lab	Status
Reconnaissance	T1595.001	Active Scanning: Scanning IP Blocks	Windows Firewall DROP log via Splunk (WinFirewallLog)	Detected
Discovery	T1046	Network Service Discovery	Same nmap targeted port scan via WinFirewallLog	Detected
Credential Access	T1110.001	Brute Force: Password Guessing	Splunk 4625 events with fabricated credentials	Detected
Initial Access	T1078	Valid Accounts	Splunk 4624 successful logon monitoring	Detectable
Execution	T1059	Command and Scripting Interpreter	Sysmon Event ID 1 process creation → Splunk + Elastic	Detectable
Execution	T1204	User Execution	Sysmon Event ID 1 with parent-process metadata	Detectable
Persistence	T1543.003	Create or Modify System Process: Windows Service	Elastic ML job v3_windows_anomalous_service_ea	Detectable
Persistence	T1053.005	Scheduled Task/Job: Scheduled Task	Sysmon Event ID 1 for schtasks.exe	Detectable
Defense Evasion	T1055	Process Injection	Sysmon Events 8, 10 (CreateRemoteThread, ProcessAccess)	Detectable
Lateral Movement	T1021.001	Remote Services: RDP	Elastic ML v3_windows_rare_user_type10_remote_login_ea	Detectable
Command and Control	T1071	Application Layer Protocol	Elastic ML v3_windows_anomalous_network_activity_ea (destination.ip field missing)	Partial

Coverage summary: 3 techniques Detected (with captured evidence), 7 Detectable (log source present), 1 Partial (known field-mapping gap on destination.ip). Full production coverage would require me to run additional test traffic to move Detectable entries into Detected, plus fix the ECS field mapping to close the Partial entry.

10.5 D3FEND Countermeasure Mapping

D3FEND is MITRE's defensive counterpart to ATT&CK. The following table maps the controls implemented in this environment against D3FEND techniques:

D3FEND Tactic	Technique ID	Countermeasure	Control Implemented in Lab	Status
Detect	D3-NTA	Network Traffic Analysis	Splunk ingestion of WinFirewallLog + WinEventLog	Implemented
Detect	D3-PA	Process Analysis	Sysmon Event ID 1 → Splunk + Elastic	Implemented
Detect	D3-UBA	User Behavior Analysis	Elastic ML v3_windows_anomalous_user_name_ea	Implemented
Detect	D3-SFA	System File Analysis	Sysmon file/hash events (SwiftOnSecurity config)	Implemented
Detect	D3-RTSD	Remote Terminal Session Detection	Elastic ML v3_windows_rare_user_type10_remote_login_ea	Implemented
Detect	D3-CIA	Credential Information Auditing	Windows Security Event 4625/4624 forwarded to Splunk	Implemented
Detect	D3-EL	Executable Analysis / Anomaly Detection	LogAI + Elastic ML anomalous process detection	Partial
Isolate	D3-IA	Identifier Analysis (IOC Enrichment)	OpenCTI 725 malware / 181 intrusion sets, not wired into SOAR playbook yet	Partial
Harden	D3-EAL	Executable Allowlisting (WDAC/AppLocker)	Not implemented, malware execution only detectable post-fact	Gap
Harden	D3-MFA	Multi-factor Authentication	Not implemented, single-factor only across lab	Gap
Harden	D3-CH	Credential Hardening (LAPS,	Not implemented, no credential auditing or rotation policy	Gap

D3FEND Tactic	Technique ID	Countermeasure	Control Implemented in Lab	Status
		rotation)		
Deceive	D3-DE	Decoy Environment / Honeypot	Not implemented, no deception layer	Gap

Coverage summary: 6 countermeasures implemented, 2 partial, 4 gaps. The partial + gap entries feed directly into the gap analysis in Section 10.7 and my prioritized recommendations in 10.8.

10.6 SOAR Playbook Execution Evidence

The completed workflow: Schedule → Splunk Search → Filter List, runs on the 5-minute cadence I configured in Node 1. Shuffle's execution history shows my recurring runs at each scheduled interval, with the Splunk action returning HTTP 200 responses containing EventCode field data. That covers the rubric requirement of at least one automated SOAR playbook built and triggered.

My Filter List node's configuration means when a real detection occurs, the search returns EventCode 4625, the workflow proceeds past the filter. That's the hook point where I'd extend the playbook in a production build with additional action nodes (case creation into TheHive, notification, IP block via EDR API) without changing anything upstream. The core detection loop is done; the response actions are the next layer.

10.7 Gap Analysis

Based on my coverage tables in 10.4 and 10.5, and observations across all four tasks, I've identified the following gaps. I've grouped them by category and prioritized them in Section 10.8.

1. Default-Off Firewall Logging (Detection Gap)

Windows Firewall does not log blocked connections by default. I discovered this directly during Task 3, my initial Splunk search for the attacker's IP returned zero results even though a full nmap scan had been executed against the endpoint. Every dropped packet went silently into the bin. That's a real visibility gap that would leave reconnaissance activity invisible to a SOC running on default Windows configs. **Remediation:** enable Set-NetFirewallProfile -LogBlocked True across all profiles as a baseline hardening step, ideally via GPO across the estate.

2. Sysmon Network Event Coverage (Detection Gap)

The SwiftOnSecurity Sysmon config is deliberately conservative on Event ID 3 (Network Connection) to manage event volume. During Task 3, no Sysmon network events were generated for my SYN scan the scan technique avoids completing TCP handshakes that would trigger connection-level logging. **Remediation:** augment Sysmon with a more aggressive network logging profile on high-value endpoints, or supplement with a network sensor like Zeek or Suricata for packet-level visibility.

3. Executable Allowlisting Not Implemented (Prevention Gap)

I didn't configure D3-EAL (Executable Allowlisting) via WDAC or AppLocker. Malware execution via T1059 (Command and Scripting Interpreter) or T1204 (User Execution) is only detectable after the fact via Sysmon Event ID 1, not preventable. **Remediation:** deploy WDAC in audit mode first, then enforcement, to block untrusted binaries at the endpoint.

4. Credential Hardening (Prevention Gap)

I didn't implement D3-CH (Credential Hardening), no MFA, no password rotation policy, no credential auditing. My Task 3 SMB brute-force reached the auth subsystem and only failed because the fabricated credentials didn't match a real account. In production, a valid weak password would result in successful lateral movement. **Remediation:** enforce MFA, deploy LAPS for local admin credentials, integrate Azure AD Password Protection.

5. TheHive Deployment Constrained by Memory (Operational Gap)

TheHive 5 requires Cassandra + Elasticsearch as backend services, with combined memory demands exceeding 3 GB heap even at minimum config. On my lab VM with 15 GB total RAM running Splunk, Elastic, LogAI, OpenCTI (stopped), and Shuffle concurrently, TheHive couldn't stabilize, ES failed to accept connections under tight heap allocation. That mirrors a real enterprise consideration: SOC tool stacks are memory-intensive and require capacity planning. **Remediation:** dedicate a separate VM (or physical host) with a minimum of 16 GB RAM to TheHive and its dependencies, or use a managed cloud instance.

6. Automated Response Actions Limited (SOAR Gap)

The Shuffle playbook I implemented (Schedule → Splunk Search → Filter) demonstrates automated detection and conditional logic, but doesn't yet execute containment actions automated firewall blocks, AD account disable, endpoint isolation via EDR API. This is a scope decision, not a technical limitation. I could extend the same workflow with additional action nodes. **Remediation:** integrate the deployed EDR platform (Sophos Intercept X) or Wazuh active response, and add Shuffle actions for account lockout and IP block operations.

7. Threat Intelligence Enrichment Not Wired Into Playbook (Integration Gap)

I deployed OpenCTI with 725 malware entities and 181 intrusion sets from the MITRE feed, but its threat intel isn't currently consumed by my SOAR playbook. A production playbook would enrich each detected IOC against OpenCTI's knowledge base to determine attribution and prioritization before creating a case. **Remediation:** add an OpenCTI enrichment node to the Shuffle workflow between the Splunk search and the response action.

8. Threat Hunting Automation (Process Gap)

My Task 3 hunts were manual I wrote SPL queries and pivoted across log sources by hand. There's no automated retrospective hunting for newly-published IOCs against historical log data. **Remediation:**

implement Sigma rules mapped to MITRE ATT&CK techniques, running as scheduled Splunk saved searches feeding into Shuffle.

10.8 Prioritized Recommendations

1. **Immediate (high impact, low effort):** Enable Windows Firewall blocked-connection logging as a domain-wide GPO. Extend Sysmon config for network events on privileged hosts.
2. **Short-term (30–60 days):** Deploy TheHive on adequate hardware. Wire OpenCTI enrichment into the Shuffle playbook. Add a Sigma rule pack to Splunk.
3. **Medium-term (60–180 days):** Implement WDAC in audit mode, migrate to enforcement. Deploy MFA and LAPS. Extend Sophos Intercept X integration into Shuffle for API-driven response.
4. **Long-term:** Consider deception (D3-DE) honeypot deployment for early-warning detection of insider threats and lateral movement.

10.9 Task 4 Conclusion

Task 4 closes my AI-Augmented SOC implementation by adding an automated response layer on top of the detection and threat-intel work from Tasks 1–3. My Shuffle playbook is a working SOAR loop, scheduled trigger through SIEM query to conditional filtering tied directly to the failed-logon attack scenario I ran manually in Task 3. That gives the whole project narrative continuity across all four tasks.

My MITRE ATT&CK and D3FEND mappings give structured views of what my environment covers and what it doesn't. The gap analysis translates the lab limitations I observed into prioritized production recommendations. TheHive's deployment challenge is itself a legitimate finding, SOC tool stacks require dedicated capacity planning, and infrastructure constraints are worth documenting rather than hiding.

Decisions Log

Key architectural and engineering decisions I made across all four tasks, with rationale. I'm including this as a reference for anyone reproducing or extending my work.

Decision	Rationale
Pass --run-as-root to Splunk explicitly	Splunk 10.4.1 aborts silently when running as root without the flag. Explicit acknowledgement was cleaner than restructuring for a dedicated splunk user mid-install.
Install Elastic Stack natively via APT rather than Docker	Fewer layers to reason about. I didn't want container troubleshooting stacked on top of Elastic troubleshooting for a lab deployment.
Elastic Agent in standalone mode, no Fleet Server	Single-agent lab, no point standing up Fleet Server infrastructure. The standalone policy YAML with embedded API key installs directly on the endpoint.
Run both SIEMs (Splunk + Elastic) in parallel	The rubric called for it. Also useful in practice, Splunk had more mature ingestion tooling for Windows event logs; Elastic ML gave me prebuilt Windows anomaly detection jobs Splunk didn't match without additional paid features.
Least-privilege for the Universal Forwarder, add to Event Log Readers group instead of running as Local System	Fixes the Sysmon channel access-denied issue without granting the forwarder broader system rights. Better hygiene by default.
LogAI in an isolated Conda Python 3.10 environment	Ubuntu 26.04 ships Python 3.14, which broke LogAI's gensim C-extension compilation. Isolation let me avoid touching system Python entirely.
Set LogAI One-Class SVM nu=0.1 via OneClassSVMParams object	Default nu=0.5 flagged 45% of buckets, a sensitivity artefact, not real detection. LogAI's factory pattern requires the typed params object, not a dict.
Remap Shuffle OpenSearch to host port 9201	Native Elasticsearch from Task 1 owned 9200. Container-internal port stayed the same so no downstream Shuffle config changes were needed.
Shuffle Schedule trigger over Webhook trigger	Webhook would require additional Splunk alert-action config on the SIEM side, which tests my Splunk alerting rather than SOAR concepts. Polling isolates automation logic to Shuffle and demonstrates the core SOAR pattern.
Use earliest_time=-24h during Shuffle playbook testing	My Task 3 failed-logon events happened hours before my Task 4 test runs. -24h window ensured they'd still be in scope during validation. Production would tighten to -5m.
Document TheHive as a capacity gap rather than	Available RAM after Shuffle was around 3.6 GiB, below what

Decision	Rationale
debug further	Cassandra + ES + TheHive JVM need to run stably. Further tuning wasn't viable on my lab host. A legitimate SOC capacity-planning finding rather than a functional failure.
Enable Windows Firewall blocked-connection logging explicitly	Default-off logging silently discarded my entire Task 3 scan traffic. Enabling it exposed 18 DROP events that my initial Splunk search would otherwise have missed entirely.